

SQL + Python Interview Prep Set

12 SQL questions | 12 Python questions | Frequently asked, product-company interview level

SECTION A — SQL QUESTIONS

Q1. Nth Highest Salary (Without LIMIT)

Window Functions | DENSE_RANK

Problem Statement:

Given an Employee table, write a query to find the 2nd highest salary without using LIMIT or OFFSET. Generalize it so it can find the Nth highest salary using a window function.

Schema:

Column	Type
emp_id	INT
name	VARCHAR
department	VARCHAR
salary	INT

Sample Input:

```
emp_id | name | department | salary
1 | Arjun | Sales | 55000
2 | Priya | Sales | 62000
3 | Rohit | Tech | 71000
4 | Kavya | Tech | 71000
5 | Meena | HR | 48000
```

Expected Output:

2nd highest salary = 71000 (note: two employees tie for highest, so DENSE_RANK, not ROW_NUMBER, is required to get the correct 2nd distinct value)

Approach Hint:

Use DENSE_RANK() OVER (ORDER BY salary DESC), then filter WHERE rn = 2. Explain why DENSE_RANK is used over RANK or ROW_NUMBER when duplicates exist.

Q2. Department-wise Highest Earners

Window Functions | PARTITION BY

Problem Statement:

For each department, find the employee(s) with the highest salary. If there's a tie within a department, return all tied employees.

Schema:

Column	Type
emp_id	INT
name	VARCHAR
department	VARCHAR
salary	INT

Sample Input:

Same Employee table as Q1

Expected Output:

Tech: Rohit (71000), Kavya (71000); Sales: Priya (62000); HR: Meena (48000)

Approach Hint:

Use RANK() OVER (PARTITION BY department ORDER BY salary DESC) and filter `rnk = 1`. Explain why PARTITION BY resets the ranking per group.

Q3. Employees Earning More Than Their Manager

Self Join

Problem Statement:

Given an Employee table with a `manager_id` column referencing another employee's `emp_id`, find all employees who earn more than their direct manager.

Schema:

Column	Type
<code>emp_id</code>	INT
<code>name</code>	VARCHAR
<code>salary</code>	INT
<code>manager_id</code>	INT

Sample Input:

```
emp_id | name | salary | manager_id
1 | Aman | 90000 | NULL
2 | Ritu | 75000 | 1
3 | Sunil | 95000 | 1
4 | Devi | 60000 | 2
```

Expected Output:

Sunil (95000) earns more than manager Aman (90000)

Approach Hint:

Self-join the table on `e.manager_id = m.emp_id`, then filter `WHERE e.salary > m.salary`. Handle the case where `manager_id` IS NULL (top-level employees) using an INNER JOIN so they're naturally excluded.

Q4. Find Duplicate Rows in a Table

GROUP BY | HAVING

Problem Statement:

Given a Customers table that may have duplicate entries based on email, write a query to identify all duplicate emails and how many times each appears.

Schema:

Column	Type
<code>cust_id</code>	INT
<code>name</code>	VARCHAR
<code>email</code>	VARCHAR

Sample Input:

```

cust_id | name | email
1 | Neha | neha@mail.com
2 | Rahul | rahul@mail.com
3 | Neha2 | neha@mail.com
4 | Sam | sam@mail.com

```

Expected Output:

email = neha@mail.com, count = 2

Approach Hint:

GROUP BY email HAVING COUNT(*) > 1. Follow-up: write a second query that also deletes duplicates while keeping the earliest cust_id using a subquery or window function.

Q5. Consecutive Days Login Streak (Gaps and Islands)

Gaps and Islands | Window Functions

Problem Statement:

Given a table of user login dates, find users who logged in on at least 3 consecutive calendar days.

Schema:

Column	Type
user_id	INT
login_date	DATE

Sample Input:

```

user_id | login_date
1 | 2026-01-01
1 | 2026-01-02
1 | 2026-01-03
1 | 2026-01-05
2 | 2026-01-01

```

Expected Output:

user_id = 1 qualifies (Jan 1, 2, 3 are 3 consecutive days). user_id = 2 does not.

Approach Hint:

Classic gaps-and-islands trick: use ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date), then compute (login_date - ROW_NUMBER) as a 'group key' — consecutive dates will produce the same group key. Then GROUP BY user_id, group_key HAVING COUNT(*) >= 3.

Q6. Running Total / Cumulative Revenue by Month

Window Functions | Running Aggregates

Problem Statement:

Given monthly revenue per region, compute the cumulative (running) revenue for each region ordered by month.

Schema:

Column	Type
region	VARCHAR
month	DATE
revenue	INT

Sample Input:

```

region | month | revenue
North | 2026-01-01 | 10000
North | 2026-02-01 | 15000
North | 2026-03-01 | 12000

```

Expected Output:

North: Jan cumulative=10000, Feb cumulative=25000, Mar cumulative=37000

Approach Hint:

SUM(revenue) OVER (PARTITION BY region ORDER BY month ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW). Be ready to explain the difference between ROWS and RANGE framing.

Q7. Top N Products per Region

Window Functions | ROW_NUMBER

Problem Statement:

Given a Sales table, find the top 3 best-selling products (by total quantity sold) in each region.

Schema:

Column	Type
region	VARCHAR
product	VARCHAR
quantity	INT

Sample Input:

```

region | product | quantity
North | A | 120
North | B | 300
North | C | 90
North | D | 250

```

Expected Output:

North top 3 by quantity: B(300), D(250), A(120)

Approach Hint:

First SUM(quantity) GROUP BY region, product, then wrap in a CTE and apply ROW_NUMBER() OVER (PARTITION BY region ORDER BY total_qty DESC), filter rn <= 3.

Q8. Customers Who Never Placed an Order

LEFT JOIN | NOT EXISTS

Problem Statement:

Given Customers and Orders tables, find all customers who have never placed an order. Solve it two ways: using LEFT JOIN and using NOT EXISTS, and be ready to discuss performance differences.

Schema:

Column	Type
cust_id	INT
name	VARCHAR

Sample Input:

```

Customers: (1,Anu) (2,Vikram) (3,Sara)
Orders: (101, cust_id=1) (102, cust_id=1)

```

Expected Output:

Vikram and Sara never placed an order

Approach Hint:

LEFT JOIN Orders ON cust_id, WHERE o.order_id IS NULL. Alternative: WHERE NOT EXISTS (SELECT 1 FROM Orders o WHERE o.cust_id = c.cust_id). Discuss when NOT EXISTS outperforms LEFT JOIN + IS NULL on large tables (short-circuiting).

Q9. Year-over-Year / Month-over-Month Growth

LAG / LEAD

Problem Statement:

Given monthly sales figures, compute the month-over-month percentage growth for each month.

Schema:

Column	Type
month	DATE
sales	INT

Sample Input:

```
month | sales
2026-01-01 | 10000
2026-02-01 | 12000
2026-03-01 | 9000
```

Expected Output:

Feb growth = $(12000 - 10000) / 10000 = 20\%$; Mar growth = $(9000 - 12000) / 12000 = -25\%$

Approach Hint:

Use LAG(sales) OVER (ORDER BY month) to get previous month's value, then compute $(\text{sales} - \text{prev_sales}) * 100.0 / \text{prev_sales}$. Watch for divide-by-zero / NULL handling on the first row.

Q10. Pivot Rows to Columns Without PIVOT

Conditional Aggregation

Problem Statement:

Given a table of monthly sales per product in long format, transform it into a wide format with one column per month, using only standard SQL (no database-specific PIVOT function).

Schema:

Column	Type
product	VARCHAR
month	VARCHAR
sales	INT

Sample Input:

```
product | month | sales
Widget | Jan | 500
Widget | Feb | 700
Gadget | Jan | 300
```

Expected Output:

product | Jan | Feb Widget | 500 | 700 Gadget | 300 | NULL (or 0)

Approach Hint:

SUM(CASE WHEN month = 'Jan' THEN sales ELSE 0 END) AS Jan, repeated per month column, GROUP BY product. Explain why this generalizes across any SQL engine, unlike vendor-specific PIVOT syntax.

Q11. Median Salary Without a Built-in Median Function

Percentile | Window Functions

Problem Statement:

Compute the median salary across all employees without relying on a database's built-in MEDIAN or PERCENTILE_CONT function (assume it's unavailable). Handle both even and odd row counts.

Schema:

Column	Type
emp_id	INT
salary	INT

Sample Input:

```
salaries: 40000, 50000, 60000, 70000
```

Expected Output:

Median = (50000 + 60000) / 2 = 55000

Approach Hint:

Use ROW_NUMBER() twice (ascending and descending order) or two window ranks, then average the middle one/two rows where the row numbers satisfy the midpoint condition based on total COUNT(*).

Q12. Most Recent Record per Group

Correlated Subquery | Window Functions

Problem Statement:

Given an order status history table where each order can have multiple status updates, find the latest status of each order.

Schema:

Column	Type
order_id	INT
status	VARCHAR
updated_at	TIMESTAMP

Sample Input:

```
order_id | status | updated_at
1001 | Placed | 2026-01-01 10:00
1001 | Shipped | 2026-01-02 14:00
1001 | Delivered | 2026-01-04 09:00
1002 | Placed | 2026-01-02 11:00
```

Expected Output:

order 1001 -> Delivered (2026-01-04 09:00); order 1002 -> Placed

Approach Hint:

Use ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY updated_at DESC) in a CTE, filter rn = 1. Alternative: correlated subquery with MAX(updated_at). Discuss which approach performs better at scale.

SECTION B — PYTHON QUESTIONS

Q1. Two Sum

Arrays | Hashmap

Problem Statement:

Given an array of integers `nums` and an integer `target`, return the indices of the two numbers such that they add up to target. Assume exactly one solution exists, and you may not use the same element twice.

Sample Input:

```
nums = [2, 7, 11, 15], target = 9
```

Expected Output:

[0, 1] (because $\text{nums}[0] + \text{nums}[1] = 2 + 7 = 9$)

Approach Hint:

Use a single-pass hashmap storing {value: index} as you iterate; for each number check if (target - number) already exists in the map. Achieves $O(n)$ time instead of the $O(n^2)$ brute force.

Requirements:

Write it as a function `two_sum(nums: list[int], target: int) -> list[int]` with a docstring and time/space complexity noted in a comment.

Q2. Reverse a Linked List

Linked List

Problem Statement:

Given the head of a singly linked list, reverse the list and return the new head.

Sample Input:

```
1 -> 2 -> 3 -> 4 -> None
```

Expected Output:

4 -> 3 -> 2 -> 1 -> None

Approach Hint:

Iterative approach using three pointers (prev, curr, next_node) is preferred over recursive for $O(1)$ space. Walk through both approaches and be ready to explain the trade-off.

Requirements:

Define a simple Node class first, then implement `reverse_list(head: Node) -> Node`. Include both iterative and recursive versions.

Q3. Valid Parentheses

Stack

Problem Statement:

Given a string containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid (every open bracket is closed by the same type in the correct order).

Sample Input:

```
s = "{[()]}" -> valid
s = "{[(])}" -> invalid
```

Expected Output:

True for the first example, False for the second

Approach Hint:

Use a stack: push opening brackets, and on a closing bracket, check if the top of the stack matches its corresponding opener; pop if it does, else return False. String is valid if the stack is empty at the end.

Requirements:

Function ``is_valid(s: str) -> bool``. Handle edge cases: empty string, single unmatched bracket, and odd-length strings.

Q4. Merge Two Sorted Lists/Arrays

Two Pointers | Merge

Problem Statement:

Given two sorted arrays, merge them into a single sorted array without using the built-in `sort()` function.

Sample Input:

```
arr1 = [1, 3, 5, 7], arr2 = [2, 4, 6, 8]
```

Expected Output:

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

Approach Hint:

Classic two-pointer merge (same technique used in merge sort's merge step, and conceptually similar to merging sorted partitions in distributed processing).

Requirements:

Function ``merge_sorted(arr1: list[int], arr2: list[int]) -> list[int]``, $O(n+m)$ time, no built-in sort.

Q5. LRU Cache (Design)

OOP Design | Hashmap + Doubly Linked List

Problem Statement:

Design and implement a Least Recently Used (LRU) Cache with a fixed capacity. It should support `get(key)` and `put(key, value)` in $O(1)$ average time each. When the cache exceeds capacity, evict the least recently used item.

Sample Input:

```
cache = LRUCache(2)
cache.put(1, 'a')
cache.put(2, 'b')
cache.get(1) # returns 'a', marks 1 as recently used
cache.put(3, 'c') # evicts key 2 (least recently used)
cache.get(2) # returns None (evicted)
```

Expected Output:

```
get(1) -> 'a', put(3,'c') evicts key 2, get(2) -> None
```

Approach Hint:

Combine a hashmap (for $O(1)$ lookup) with a doubly linked list (for $O(1)$ reordering of recency). Python's `OrderedDict` can also do this in an interview if you explain the underlying mechanics.

Requirements:

Implement class ``LRUCache`` with ``__init__(self, capacity: int)``, ``get(self, key)``, ``put(self, key, value)``. Include docstrings and explain time complexity for each method.

Q6. Find Missing Number in Array

Arrays | Math

Problem Statement:

Given an array containing n distinct numbers taken from the range $[0, n]$, find the one number that is missing from the array.

Sample Input:

```
nums = [3, 0, 1] (n = 3, range is 0..3)
```

Expected Output:

2

Approach Hint:

Use the sum formula: $\text{expected_sum} = n*(n+1)/2$, then $\text{missing} = \text{expected_sum} - \text{actual_sum}$. Alternative: XOR-based approach for integer overflow-safe environments — be ready to explain both.

Requirements:

Function `find_missing(nums: list[int]) -> int``, $O(n)$ time, $O(1)$ space.

Q7. Group Anagrams

Hashmap | Strings

Problem Statement:

Given an array of strings, group the anagrams together. Return the groups in any order.

Sample Input:

```
words = ['eat', 'tea', 'tan', 'ate', 'nat', 'bat']
```

Expected Output:

```
[['eat','tea','ate'], ['tan','nat'], ['bat']]
```

Approach Hint:

Use a hashmap where the key is the sorted tuple of characters (or a character-count signature) of each word, and the value is the list of original words sharing that signature. This mirrors real data-grouping logic used in ETL aggregation steps.

Requirements:

Function `group_anagrams(words: list[str]) -> list[list[str]]``. Discuss why sorting each word is $O(k \log k)$ and whether a character-count tuple key is more efficient for long words.

Q8. Longest Substring Without Repeating Characters

Sliding Window

Problem Statement:

Given a string, find the length of the longest substring without repeating characters.

Sample Input:

```
s = "abcabcbb"
```

Expected Output:

3 (the substring "abc" has length 3)

Approach Hint:

Sliding window with a hashmap/set tracking characters currently in the window; when a repeat is found, shrink the window from the left until the duplicate is removed.

Requirements:

Function `longest_unique_substring(s: str) -> int``, $O(n)$ time using the sliding window technique (avoid the $O(n^2)$ brute force).

Q9. Flatten a Nested List/Dictionary

Problem Statement:

Given an arbitrarily nested list (or dictionary, e.g. nested JSON), write a function to flatten it into a single flat list (or a flat dictionary with dotted keys for the dict version).

Sample Input:

```
nested_list = [1, [2, [3, 4], 5], 6]
nested_dict = {'a': 1, 'b': {'c': 2, 'd': {'e': 3}}}
```

Expected Output:

```
flat_list = [1, 2, 3, 4, 5, 6]
flat_dict = {'a': 1, 'b.c': 2, 'b.d.e': 3}
```

Approach Hint:

Recursive solution: check the type of each element; if it's a list/dict, recurse into it and extend/merge results, otherwise append/assign directly. This directly mirrors real-world JSON/semi-structured data parsing in ETL pipelines.

Requirements:

Implement both `flatten\_list(nested: list) -> list` and `flatten\_dict(nested: dict, parent\_key: str = "") -> dict`.

Q10. Custom Retry Decorator

Problem Statement:

Write a decorator `@retry(times=3, delay=1)` that retries a function call up to `times` times if it raises an exception, waiting `delay` seconds between attempts. If all attempts fail, re-raise the last exception.

Sample Input:

```
@retry(times=3, delay=1)
def flaky_call():
    # simulates a call that fails randomly
    ...
```

Expected Output:

flaky_call() is retried up to 3 times before the exception propagates if it keeps failing

Approach Hint:

A decorator factory: outer function takes (times, delay) as arguments, returns the actual decorator, which wraps the target function in a try/except loop. Use functools.wraps to preserve the original function's metadata.

Requirements:

Implement `retry(times: int = 3, delay: int = 1)` as a decorator factory. Include proper exception handling and logging of each failed attempt.

Q11. Generator-Based Lazy Chunk Reader

Problem Statement:

Write a generator function that reads a large file (or list) in fixed-size chunks, yielding one chunk at a time instead of loading everything into memory at once.

Sample Input:

```
data = list(range(1, 23)) # simulate a large dataset
chunk_size = 5
```

Expected Output:

Yields: [1,2,3,4,5], [6,7,8,9,10], [11,12,13,14,15], [16,17,18,19,20], [21,22]

Approach Hint:

Use `yield` inside a loop that slices the iterable by `chunk_size`, or for real files, read `f.readlines(chunk_size)` in a loop. Explain why this is memory-efficient compared to loading the full dataset into a list first — directly relevant to how Spark/pipeline code processes data in partitions.

Requirements:

Function `chunk_reader(iterable, chunk_size: int)` implemented as a generator (using `yield`, not returning a list of lists).

Q12. Merge Overlapping Intervals

Sorting | Intervals

Problem Statement:

Given a list of intervals where each interval is `[start, end]`, merge all overlapping intervals and return a list of the non-overlapping intervals that cover all the ranges in the input.

Sample Input:

```
intervals = [[1,3], [2,6], [8,10], [15,18]]
```

Expected Output:

```
[[1,6], [8,10], [15,18]]
```

Approach Hint:

Sort intervals by start time first. Then iterate through, and if the current interval's start is $\leq$ the end of the last merged interval, merge them (extend the end); otherwise, start a new interval. This pattern is directly used in real pipeline work for deduplicating overlapping time windows.

Requirements:

Function `merge_intervals(intervals: list[list[int]]) -> list[list[int]]`, $O(n \log n)$ due to the sort step.